

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Low (Pseudo-code Writing)

Course Code: CSA0501 **Course Title:** Programming in Java

CO Assessed: CO1 – Precision (BL3,BL4) **Assessment:** Assessment Tool 3 - Pseudo-code Writing

Weightage: **Total Marks:** 100

CO1 Statement: Apply object-oriented programming principles such as classes, objects encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)

Student Name: PRIYA MITHRA S **Reg. No.:** 1924260060

Section Batch: / SLOT B **Date:** 22.07.26

Code of Conduct

I PRIYA MITHRA.S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 90 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation 20
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach 30
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs 20
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear 60
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient 16

96
100

96%

PSEUDO-CODE WRITING

Q1.

STUDENT MANAGEMENT SYSTEM

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details.
- Calculate average marks.
- Display result (Pass/Fail).

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP concepts: Encapsulation | Data Hiding | Classes & Objects

CLASS STUDENT

// Private attributes (Data hiding)

Private name

Private rollNo

Private marks[3]

// Method to accept student details.

Public Method acceptDetails()

Input name

Input rollNo

For i=0 to 2

Input marks[i]

End Method

// Method to calculate average marks

public Method calculateAverage()

total = 0

For i=0 to 2

total = total + marks[i]

End for

average = total / 3

Return average

End Method

// Method to display result

public Method displayResult()

average = calculateAverage()

print "Name:", name

print "Roll No:", rollNo

print "Average Marks:", average

If average >= 40 then

Print "Result: PASS"

Else

Print "Result: FAIL"

End If

End Method

END CLASS

START //main Program

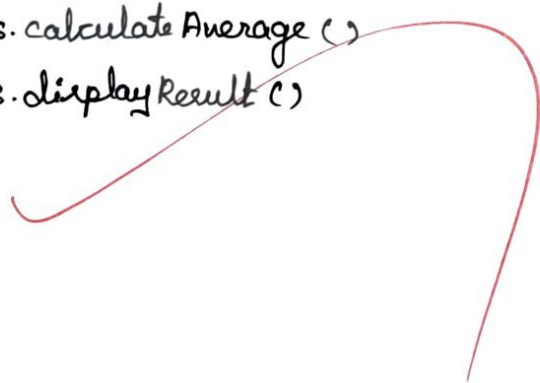
DECLARE S : Student

s.acceptDetails()

s.calculateAverage()

s.displayResult()

STOP



Q2.

LIBRARY BOOK MANAGEMENT

Design a book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status).
- Return a book
- Display book details using getter methods.

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP concepts: Encapsulation | Getters & Setters | Objects

CLASS BOOK

// Private attributes (Data Hiding)

Private title

Private author

Private ISBN

Private isAvailable

// constructor

Public Method Book (t, a, i)

title = t

author = a

ISBN = i

isAvailable = True

End Method

// Getter methods

Public Method getTitle()

Return title

End Method

Public Method getAuthor()

Return author

End Method

Public Method getISBN()

Return ISBN

End Method

Public Method getAvailability()

Return isAvailable

End Method

// Issue a book

Public Method issueBook()

If isAvailable = True Then

isAvailable = False

Print "Book issued successfully."

Else

Print "Book is already issued."

End If

End Method

// return a book

Public Method returnBook()

If isAvailable = False Then

isAvailable = True

Print "Book returned successfully."

Else

Print "Book is already available."

End If.

End Method.

// Display book details

Public Method displayDetails()

print "Title: ", getTitle()

print "Author: ", getAuthor()

print "ISBN:", getISBN()

print "Available: ", getAvailability()

End Method

END CLASS

MAIN PROGRAM

// create book objects

Create book 1 = Book("Python Basics", "John Smith", "ISBN101")

Create book 2 = Book("Java programming", "Alice Brown", "ISBN102")

Create book 3 = Book("Data Structures", "David Lee", "ISBN103")

// Display details of all books

book 1. displayDetails()

book 2. displayDetails()

book 3. displayDetails()

// Simulate issue and return operations

1. Issue book 1

book 1. issueBook()

book 1. displayDetails()

2. Return book 1

book 1. returnBook()

book 1. displayDetails()

3. Issue book 2

book 2. issueBook()

book 2. displayDetails()

Q3.

EMPLOYEE SALARY CALCULATION

write pseudocode for an employee class with encapsulated attributes (empid, name, basicpay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP concepts: Encapsulation | Data hiding | Methods.

CLASS EMPLOYEE:

// private attributes (data hiding)

private empid

private name

private basicpay

private designation

// Method to accept employee details

Public Method accept details()

input empid

input name

input Basicpay

input designation

End Method

// Method to calculate Gross Salary.

public Method calculateGross salary()

gross salary = calculateGrossSalary()

PF = basicpay * 0.12

// 12% Basic pay

tax = gross salary * 0.05

// 5% tax

net salary = gross salary - (PF + tax)

return netSalary

End Method

// Method to display pay slip

public Method displayPaySlip()

print "----- PAY SLIP-----"

print "Employee ID:", empId

print "Name :", name

print "Designation:", designation

print "Basic Pay :", basicPay

print "Gross Salary:", CalculateGrossSalary()

print "Net Salary: ", CalculateNetSalary()

End Method

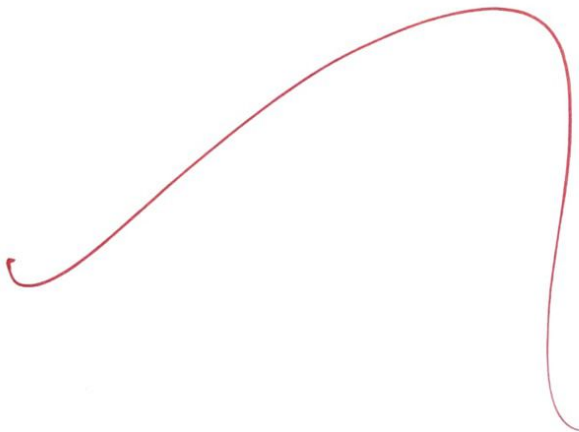
End class

// main program

Create object emp1 of Employee

emp1.acceptDetails()

emp1.displayPaySlip()



Q4. Shape Area Calculator Using Polymorphism

Create a shape base class with an overridable `calculateArea()` method. Apply this to:

- Circle - area using radius
- Rectangle - area using length and breadth
- Triangle - area using base and height

Use method overriding to demonstrate runtime polymorphism. call `calculateArea()` for each object.

OOP concepts: inheritance | polymorphism | method Overriding

CLASS SHAPE

// Overridable Method

public Method `calculateArea()`

print "Area cannot be calculated"

End Method

End class

class Circle inherits Shape

Private radius

Public Method `Circle(r)`

radius = r

End Method

// Method Overriding

Public Method `CalculateArea()`

area = $3.14 * \text{radius} * \text{radius}$

print "Area of Circle =", area

End Method

End class

25) VEHICLE REGISTRATION SYSTEM.

class Rectangle inherits Shape

Private length

Private breadth

Public Method Rectangle(l, b)

length = l

breadth = b

End Method

// Method Overriding

Public Method calculateArea()

area = length * breadth

print "Area of Rectangle =", area

End Method

End class

class Triangle inherits Shape

Private base

private height

public Method Triangle(b, h)

base = b

height = h

End Method

End class

Declare shape s // Main Program

s = New Circle(7)

s.calculateArea()

s = New Rectangle(10, 5)

s.calculateArea()

s = New Triangle(8, 6)

s.calculateArea()

Q5. VEHICLE REGISTRATION SYSTEM

Design a vehicle class with attributes (regNo, owner, fuelType)

Extend it into :

- Two wheeler - with engine capacity
- Four wheeler - with Seating capacity and AC Status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP concepts: inheritance | method overriding | Subclass

CLASS VEHICLE

//common attributes

protected regNo

protected owner

protected fuelType

// constructor

Public Method Vehicle (r, o, f)

regNo = r

owner = o

fuelType = f

End Method

//overridable method

Public Method displayDetails()

Print "Registration No :", regNo

print "Owner :", owner

print "Fuel Type :", fuel Type

End Method.

End class

class TwoWheeler inherits Vehicle

Private engineCapacity

// constructor

Public Method TwoWheeler (r, o, f, e)

vehicle (r, o, f)

engine capacity = e

End Method

// Method Overriding

Public Method displayDetails()

print "---- Two Wheeler Details ----"

print "Registration No :", regNo

print "Owner :", owner

print "Fuel Type :", fuelType

print "Engine Capacity :", engineCapacity, "cc"

End Method

class FourWheeler inherits Vehicle

Private seatingCapacity

Private acStatus

// constructor

public Method FourWheeler (r, o, f, s, ac)

vehicle (r, o, f)

seatingCapacity = s

acStatus = ac

End Method

// Method overriding

Public Method displayDetails()

print "---- four wheeler Details ----"

print "Registration number: ", regNo

print "Owner : ", owner

print "Fuel Type : ", fuelType

print "seating capacity: ", seating capacity

print "AC Status : ", acStatus

END Method

End class

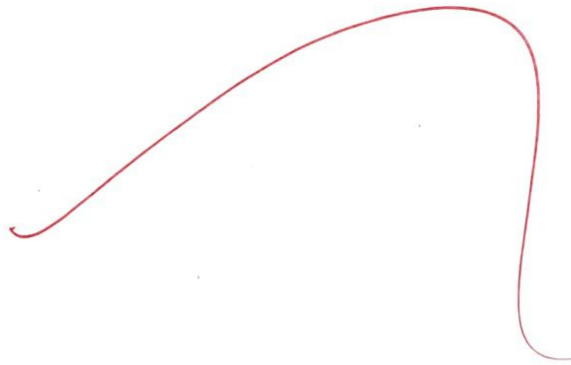
// Main Program

Create bike = New TwoWheeler ("TN01AB1234", "Rahul", "Petrol", 150)

Create car = New FourWheeler ("TN02CD5678", "Divya", "Diesel", 5, "yes")

bike.displayDetails()

car.displayDetails()



Q6.

BANK ACCOUNT HIERARCHY

Develop pseudocode for a BankAccount superclass and two subclasses - SavingsAccount (with interest calculation) and Current Account (with overdraft facility). Analyse and implement:

- Inheritance of common attributes (accNo, balance, Holder)
- Method overriding for withdrawal behaviour
- How overdraft protection differs from savings limit.

compare the behaviour of withdraw() across both subclasses and justify the design choices.

OOP concepts: Inheritance | Method Overriding | Polymorphism

CLASS BANKACCOUNT

//common Attributes

protected accNo

protected holder

protected balance

//constructor

Public Method BankAccounts (a, h, b)

accNo = a

holder = h

balance = b

END Method

//Deposit Method

Public Method deposit (amount)

balance = balance + amount

print "Amount Deposited:", amount

End Method

//overridable withdrawal method

Public Method withdraw(amount)

print "withdrawal method to be overridden."

End Method

// display account Details

Public Method displayDetails()

print "Account No:", accNo

print "Holder :", Holder

print "Balance :", balance

END Method

End class

Class SavingsAccount inherits BankAccount

Private InterestRate

//constructor

Public Method SavingsAccount(a, h, b, rate)

BankAccounts(a, h, b)

InterestRate = rate

END Method

//interest calculation

Public Method calculateInterest()

interest = balance * InterestRate / 100

print "interest = ", interest

End Method

//method Overriding

Public method withdraw (amount)

if amount $\leq$ balance then

balance = balance - amount

print "Withdrawal Successful."

Else

print "Insufficient Balance. Withdrawal Denied."

End If

End Method

End CLASS

class CurrentAccount inherits BankAccount

Private overdraftLimit

//constructor

Public Method CurrentAccounts (a, h, b, limit)

BankAccount (a, h, b)

OverdraftLimit = limit

END METHOD

//method overriding

Public Method withdraw (amount)

if amount $\leq$ (balance + overdraftLimit) then

balance = balance - amount

print "Withdrawal Successful using Overdraft if required!"

Else

print "Overdraft Limit exceeded. Withdrawal Denied!"

END If

END method

END class

//main program

Declare BankAccount account

account = New Savings Account ("SB101", "Suvidha", 10000, 5)

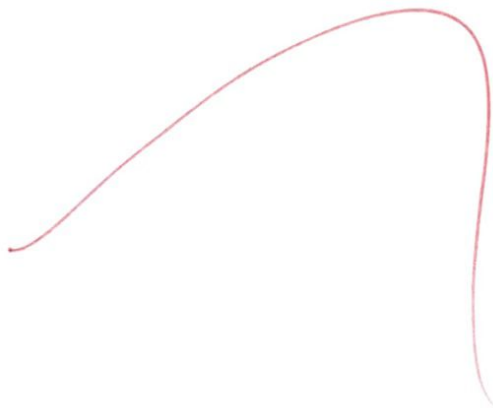
account.withdraw (12000)

account.displayDetails ()

account = New Current Account ("CA201", "Rahul", 10000, 5000)

account.withdraw (12000)

account.displayDetails ()



Q7. HOSPITAL PATIENT RECORD SYSTEM

Design a patient base class and analyse how it extends into:

- In patient - with ward number, admission date and daily charges
- Out patient - with appointment date and consultation fee.

Analyse encapsulation requirements for sensitive data (diagnosis, prescription).
Override a generateBill() method for each type and compare the billing logic differences. OOPS concepts: Encapsulation | inheritance | Polymorphism

CLASS Patient

// common attributes

protected patientId

protected name

protected age

// sensitive data (Encapsulation)

private diagnosis

private prescription

// constructor

Public Method patient (id, n, a, d, p)

patientId = id

name = n

age = a

diagnosis = d

prescription = p

End Method

// getter methods for sensitive data

Public Method getDiagnosis()

return diagnosis

End Method

Public Method getPrescription()

Return Prescription

End Method

//overridable method

Public Method generateBill()

print "Bill cannot be generated."

End Method

// Display common details

Public Method displayDetails()

print "Patient ID :", patientId

print "Name :", name

print "Age :", age

End method

End class

class InPatient inherits patient

Private wardNo

Private admissionDate

Private dailyCharges

Private daysAdmitted.

//constructor

Public method InPatient (id, n, a, d, p, w, ad, charge, days)

patient (id, n, a, d, p)

wardNo = w

admissionDate = ad

dailyCharges = charge

daysAdmitted = days

End Method

// Method overriding

Public Method generatBill ()

totalBill = dailyCharges * daysAdmitted.

Print "----- In-Patient Bill -----"

Print "Ward No : ", wardNo

Print "Admission Date: ", admissionDate

Print "Total Bill : ", totalBill

End Method

End class

class OutPatient inherits Patient

Private appointmentDate

Private consultationFee

// constructor

Public Method Outpatient (id, n, a, d, p, appDate, fee)

patient (id, n, a, d, p)

appointmentDate = appDate

consultation Fee = fee

End Method

// method Overriding

Public method. generatBill ()

Print "----- Out-patient Bill -----"

Print "Appointment Date: ", appointmentDate

Print "Consultation fee: ", consultationFee

End method

End class

// main Program

Declare Patient Patient

```
patient = New InPatient ("P101", "Rahul", 35, "Fever",  
"Medicine A", "W12", "10-09-2026", 1500, 4)
```

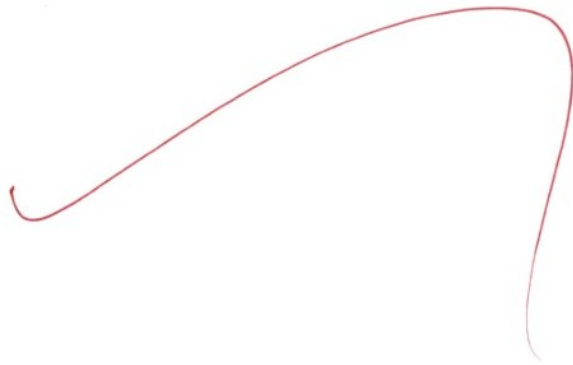
```
patient.displayDetails()
```

```
patient.generateBill()
```

```
patient = New OutPatient ("P102", "Divya", 28, "cold",  
"Medicine B", "12-09-2026", 500)
```

```
patient.displayDetails()
```

```
patient.generateBill()
```



Q8.

E - COMMERCE PRODUCT CATALOG

Analyse and design a product class hierarchy for an e-commerce System:

- Electronics - with warranty, brand, voltage
- clothing - with size, fabric, gender
- Grocery - with expiry date and weight

Override an applyDiscount() method for each category with different discount rules. Analyse why polymorphism makes the cart checkout process flexible and extensible. OOP concepts: Encapsulation | Inheritance | Polymorphism

CLASS Product

// common Attributes

protected ProductId

protected productname

protected price

// constructor

Public Method Product (id, name, p)

productId = id

productname = name

price = p

End Method

// overridable method

Public Method applyDiscount()

return price

End Method

// Display product details

Public Method

Print "Product ID : ", productId

Print "Product name:", productname

Print "Price : ", price

End Method

End class

class clothing inherits Product

private size
private fabric
private gender

// constructor

Public Method clothing (id, name, p, s, f, g)

product (id, name, p) // call parent constructor

size = s

fabric = f

gender = g

End Method

// method overriding

Public Method applyDiscount()

discountPrice = price - (price * 0.20) // 20% discount
return

End Method

End class

class electronics inherits Product

private warranty
private brand
private voltage

// constructor

Public Method Electronics (id, name, p, w, b, v)

product (id, name, p)

// call parent constructor

warranty = w

brand = b

voltage = v

End Method

// method overriding

Public Method applyDiscount()

discountPrice = price - (price * 0.10) // 10% discount

return discount Price

End Method

End class

class Grocery inherits Product

private expiryDate

private weight

Public Method Grocery (Id, name, p, e, w) *// constructor*

product (Id, name, p)

// call parent constructor

expiryDate = e

weight = w

End Method

// Method overriding

Public Method applyDiscount()

discountPrice = price - (price * 0.05) *// 5% discount*

return discountPrice

End Method

End Class

// Main Program

Declare Product item

item = New Electronics ("E101", "Laptop", 50000, "2 years",

"Dell", "230 v")

print "Electronics Price after discount = ", item.applyDiscount()

item = New clothing ("c201", "T-shirt", 1500, "M", "cotton", "unisex")

print "clothing Price after Discount = ", item.applyDiscount()

item = New Grocery ("G1301", "Rice", 1000, "30-12-2026", "5 kg")

print "Grocery Price after Discount = ", item.applyDiscount()

Q9.

UNIVERSITY STAFF HIERARCHY

Analyse the design of a Staff base class extended by Teaching Staff and nonTeaching Staff. Further extend Teaching Staff into professor and lecturers (multilevel inheritance). for each level:

- Identify which attributes should be private vs protected
- override calculate Allowance () at each level.
- Analyse how protected access enables Inheritance

without full exposure Justify

The use of multilevel Inheritance and explain how access modifiers enforce data hiding

OOPS concepts: Multilevel Inheritance | access modifiers | data Hiding

CLASS Staff

Method Calculate Allowance ()

END CLASS

CLASS Teaching Staff EXTENDS Staff

METHOD calculate Allowance ()

END CLASS

CLASS Professor EXTENDS Teaching Staff

METHOD calculate Allowance ()

END CLASS

CLASS lecturer EXTENDS Teaching Staff

METHOD calculate Allowance ()

END class

Staffs

s = Professor → calculate Allowance ()
s = lecturer → calculate Allowance () STOP

Q10

Smart Home Device Controller

Analyse and design a SmartDevice superclass with subclasses: SmartLight, smartThermostat, and smartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behaviour. Analyse:

- How polymorphism allows a single controller loop to manage all devices.

- Security implications of data hiding in smartLock (PIN must never be exposed)

Design the class hierarchy and explain the OOP principles applied at each level. OOP concepts: polymorphism | Encapsulation | Inheritance

CLASS SmartDevice

Method executeCommand()

END CLASS

CLASS SmartLight EXTENDS

SmartDevice

Method executeCommand()

END CLASS

CLASS SmartThermostat Extends

SmartDevice

Method executeCommand()

END CLASS

CLASS SmartLock EXTENDS SmartDevice

Private pin

Method executeCommand()

END CLASS

START

SmartLight → executeCommand()

SmartThermostat → executeCommand()

SmartLock → executeCommand()

STOP